RabbitTClust2: Fast, Scalable, and Versatile Genome Clustering with Greedy and MST Strategies

Tong Zhang, Zekun Yin*, Xiaoming Xu, Lifeng Yan, Yang Yang, Yijie Gao, Xiaohui Duan*, Bertil Schmidt, *Senior Member, IEEE*, and Weiguo Liu*

***Abstract*— Large-scale genome clustering is essential for genome database organization, pathogen surveillance, and downstream comparative analysis, yet rapidly growing genome databases demand methods that are efficient, scalable, and flexible across diverse analysis scenarios. This paper presents RabbitTClust2, a fast, scalable, and versatile genome clustering tool based on compact sketch representations. RabbitTClust2 supports both KSSD and MinHash sketches and introduces sketch-based inverted indexes with threshold-aware pruning to reduce unnecessary distance computations. It provides two complementary clustering strategies: a Greedy mode for efficient one-off clustering on massive datasets, and an MST mode that enables reusable clustering structures for fast incremental updates and multi-threshold analysis. To further improve scalability, RabbitTClust2 includes an MPI-based distributed MST implementation for large genome collections. It also supports practical output formats, including CD-HIT-style cluster files, Newick/PHYLIP files, and representative-sketch search tables for genome assignment. Experiments on large bacterial genome datasets show that RabbitTClust2 substantially reduces runtime and memory usage while maintaining high clustering quality. It clusters an 11 TB (FASTA format) GenBank bacterial dataset within one hour on a single server and completes MST-based clustering in less than 30 minutes on 16 nodes. RabbitTClust2 also accelerates incremental and multi-threshold clustering and helps reveal potential inconsistencies between genome similarity and taxonomic labels. RabbitTClust2 is available at `https://github.com/RabbitBio/RabbitTClust`.**

***Index Terms*— Genome clustering, sketch, incremental clustering, multi-threshold analysis.**

I. Introduction

CLUSTERING genome sequences is a fundamental step in biomedical data analysis. It supports genome dereplication [2], pathogen surveillance [3], and the organization of rapidly growing genome databases [4]. With continued advances in high-throughput sequencing [5], public genome databases have expanded rapidly. By January 2025, NCBI GenBank had archived more than 2.5 million genome assemblies, while RefSeq contained approximately 395,000 genomes representing over 28,000 species [6]. Clustering genomes at this scale is difficult for two reasons. First, genomes are long, making alignment-based similarity computation expensive. Second, the number of genome pairs grows quadratically with the number of genomes, leading to high runtime and memory cost for exhaustive all-pairs comparison. As a result, time-consuming alignment-based methods [7] and many sequence clustering tools, such as CD-HIT [8], Linclust [9], and UCLUST, are not well suited for clustering millions of long genomes [10].

A wide range of alignment-free methods have been developed for large-scale genome clustering [11], [12]. Representative examples include Gclust, which uses sparse suffix arrays and maximal exact matches [13], and MeShClust3, which predicts sequence similarity from k-mer histograms using a machine-learning model [14]. Deep-learning-based methods have also been explored, including iDeLUCS [15] and CGRclust [16]. However, their evaluations have mainly focused on moderate-scale sequence collections, rather than on the TB-scale, continuously growing genome databases targeted by this work. RabbitTClust has demonstrated the effectiveness of MinHash-based sketching for large-scale genome clustering [17], [18]. Although these methods reduce the cost of genome comparison, further performance improvement is still needed as genome databases continue to grow. Moreover, large genome databases typically require more than a single clustering run. Public genome databases are continuously updated with newly sequenced genomes, creating a need for efficient incremental clustering. In addition, the choice of similarity thresholds strongly affects clustering results, because different analysis resolutions, from species-level grouping to finer intra-species or epidemiological resolution, may require different cutoffs. Therefore, clustering results often need to be evaluated across multiple thresholds. The main challenge is thus not only to accelerate a single clustering run, but also to reduce repeated computation during database updates and multi-threshold analysis.

We present RabbitTClust2, a clustering tool for large-

An earlier conference version of this work was published in IEEE BIBM 2025 [1]. This journal version substantially extends the conference paper with an additional optimized MinHash sketching option, a Greedy mode for efficient one-off clustering, additional output formats (e.g., CD-HIT-style cluster files, PHYLIP files, and representative-sketch search tables), a broader evaluation on updated large bacterial genome datasets, and an expanded analysis of within-genus species-level threshold responses and potential differences between genome similarity and taxonomic labels.

*Corresponding authors: Zekun Yin (e-mail: zekun.yin@sdu.edu.cn), Xiaohui Duan (e-mail: sunrise.duan@sdu.edu.cn), and Weiguo Liu (e-mail: weiguo.liu@sdu.edu.cn).

scale genome clustering, incremental updates, and multi-threshold analysis. RabbitTClust2 supports both KSSD [19] and MinHash [18] sketches. It uses sketch-based inverted indexes to find genomes that share sketch hashes with a query genome, and then applies threshold-aware pruning to avoid unnecessary distance computations. RabbitTClust2 provides two clustering modes. The Greedy mode is designed for fast one-off clustering for ultra-large genome datasets. The MST mode is designed for efficient incremental and multi-threshold clustering. It saves sketches and MSTs so that users can perform incremental and multi-threshold clustering without rerunning the full clustering workflow. It also helps identify cases where similarity-based clustering deviates from existing taxonomic labels: genomes with different labels may be grouped together even at strict thresholds, while genomes with the same label may remain separated even at relaxed thresholds. RabbitTClust2 further provides an MPI-based distributed MST implementation to accelerate MST construction for large-scale genome clustering. To improve usability, RabbitTClust2 supports multiple output formats, including CD-HIT-style cluster files and Newick/PHYLIP files, and retains representative-sketch search results for subsequent analyses such as new-genome assignment.

II. Methods

A. Overview of RabbitTClust2

RabbitTClust2 is designed for efficient large-scale genome clustering, multi-threshold analysis, and incremental updates (Fig. 1). It first builds a compact sketch for each genome using KSSD or MinHash (Fig. 1(a)). The two sketching methods support genome datasets with different sequence-length distributions. MinHash produces fixed-size sketches, making memory usage easy to control for both short and long sequences. KSSD uses a valid-k-mer sampling strategy and may retain too few k-mers for very short sequences, but its variable-size sketch representation is useful when sequence lengths differ greatly in a large genome database. After sketch construction, RabbitTClust2 uses an inverted index to retrieve candidates that share at least one sketch hash. Distances are computed only for these candidates, rather than for all possible comparisons. RabbitTClust2 provides two clustering modes: *greedy* and *MST*.

The greedy mode is designed for fast one-off clustering, while MST mode is designed for reusable analysis across thresholds and incremental updates. In greedy mode (Fig. 1(b)), the inverted index is maintained only over current cluster representatives. For each incoming genome, RabbitTClust2 retrieves candidate representatives from the index and assigns the genome to an existing cluster if the distance is within the threshold. If no representative satisfies the threshold, the genome becomes a new representative and is inserted into the index.

In MST mode (Fig. 1(c)), the inverted index is built over all genomes to identify candidate genome pairs. RabbitTClust2 computes distances for these pairs and applies Kruskal's algorithm to construct a *minimum spanning tree* (MST). The MST can then be cut at different distance thresholds, enabling multi-threshold clustering without rebuilding the clustering structure for each threshold. RabbitTClust2 also provides an MPI-based distributed implementation for large-scale MST construction. This mode can save and reload sketches and MSTs, allowing these intermediate results to be reused for incremental updates and multi-threshold analysis. RabbitTClust2 also outputs CD-HIT-style cluster files, Newick/PHYLIP files, and a representative-sketch search table for assigning new genomes to existing clusters.

B. Sketch Construction

RabbitTClust2 supports two algorithms for generating compact sketches from input genomes: *KSSD* and *MinHash*.

KSSD performs dimensionality reduction by retaining only k-mers mapped to a pre-selected subset of valid partitions, where the retained partitions account for a fraction $1/2^{4\cdot\mathrm{drlevel}}$ of the encoded k-mer space ($1/4096$ at the default $\mathrm{drlevel} = 3$). As a result, the KSSD sketch size scales with the number of valid k-mers in each genome instead of a fixed sketch budget. This variable-size representation is better suited to comparisons between genomes with large length differences. To reduce sketch-construction overhead, RabbitTClust2 uses direct base-to-bit k-mer encoding for KSSD sketch construction, avoiding the cost of general-purpose hashing during sketch generation. In the original KSSD workflow, an external shuffled dictionary is used to determine whether an encoded k-mer belongs to a retained valid partition. RabbitTClust2 replaces this dictionary with a compact *map dictionary* that stores only valid-partition entries, reducing random-access overhead and improving memory locality [20]. This mapping structure is implemented using a flat hash table from parallel-hashmap (`phmap`). By keeping only the valid-partition mapping needed during sketch construction, the KSSD pipeline no longer depends on external shuffled-dictionary files. However, due to its valid-partition sampling strategy, KSSD may retain very few or even no valid k-mers for extremely short sequences.

To handle such cases, RabbitTClust2 also includes MinHash as a fixed-size sketching option. MinHash provides predictable memory usage and works for both short and long sequences. To reduce MinHash construction overhead, RabbitTClust2 avoids unnecessary sorting during heap maintenance by deferring the sorting of retained MinHash values until an ordered sketch representation is required. Precomputed sketches can be reloaded with `--presketch`, which avoids rebuilding sketches and reduces the runtime of repeated analysis on large-scale genome datasets.

C. Greedy Clustering

Greedy clustering incrementally maintains a set of current representatives R. Each incoming genome g_j is assigned to an existing representative if the sketch distance satisfies a user-defined threshold τ; otherwise, g_j becomes a new representative. This strategy avoids full graph construction, but its cost grows with the number of representatives, since the j-th insertion may require up to $O(|R_j|)$ representative comparisons. The total cost is therefore $O\left(\sum_j |R_j|\right)$ and can

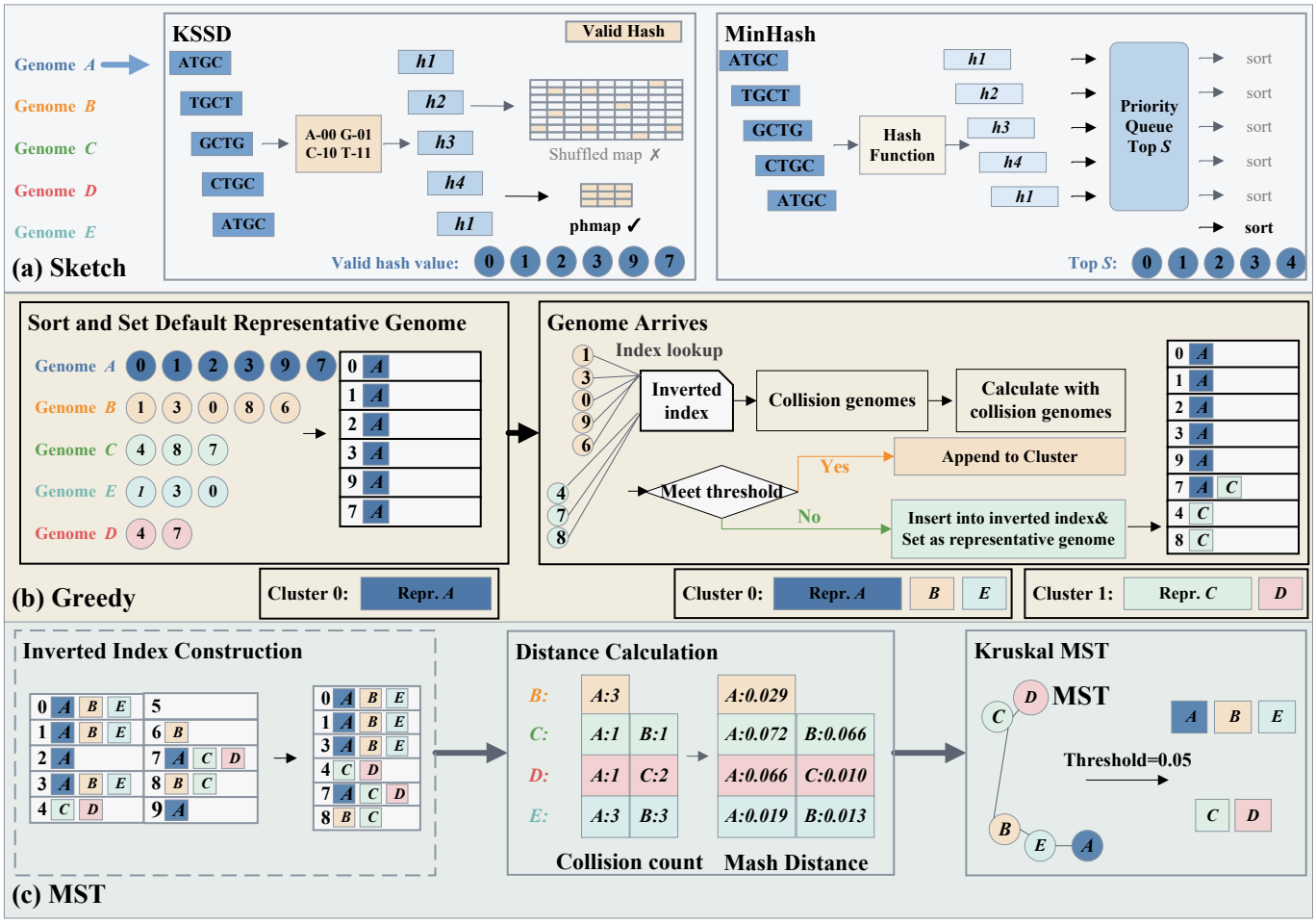


Fig. 1. Overview of RabbitTClust2. (a) KSSD or MinHash sketches are built for each genome. (b) In greedy mode, the inverted index is maintained only over current representatives, and each new genome is assigned to a candidate representative or becomes a new representative. (c) In MST mode, a dataset-wide inverted index retrieves candidate genome pairs for distance estimation, after which Kruskal's algorithm constructs an MST that can be cut at different thresholds.

approach $O(N^2)$ in the worst case. However, in large-scale genome datasets, most representative-query pairs are unlikely to satisfy the threshold, making many of these comparisons unnecessary and time-consuming. RabbitTClust2 addresses this inefficiency by adopting an index-based, threshold-aware pruning method to restrict comparisons to representative sequences that can possibly satisfy the threshold for each query genome (Fig. 1(b)).

During greedy clustering, RabbitTClust2 inserts only the sketch hashes of representative genomes into the inverted index. The first processed genome is used as the initial representative. Each hash value points to the representatives that contain this hash. Therefore, when a new genome is processed, the index can quickly identify representatives that share sketch hashes with it, rather than comparing the genome with every existing representative.

When a new genome q with sketch S_q is processed, RabbitTClust2 looks up the posting lists of the hashes in S_q. Any representative that shares at least one hash with q is treated as a candidate for assignment. For each candidate representative r, RabbitTClust2 counts the number of hashes shared with q,

$$c = |S_q \cap S_r|,$$

which is then used to estimate the sketch distance between q and r.

To avoid unnecessary distance computation, RabbitTClust2 first turns the user-specified Mash distance threshold τ into a minimum Jaccard similarity. Using the Mash distance relation

$$D = -\frac{1}{k}\ln\left(\frac{2J}{1+J}\right),$$

the condition $D \le \tau$ gives

$$J_{\min} = \frac{e^{-k\tau}}{2 - e^{-k\tau}}. \tag{1}$$

RabbitTClust2 then uses $J_{\min}$ to discard candidates that cannot satisfy the distance threshold before final assignment.

For MinHash, all sketches have the same fixed size k_s. Under the set-based sketch comparison used here, the estimated Jaccard similarity between q and r is

$$\widehat{J} = \frac{c}{2k_s - c}.$$

Because k_s is fixed, $\widehat{J}$ increases monotonically with the shared-hash count c. RabbitTClust2 therefore selects the candidate representative with the largest shared-hash count. Let

$$c_{\max} = \max_{r \in R} |S_q \cap S_r|.$$

If

$$c_{\max} \geq \left\lceil \frac{2k_s J_{\min}}{1 + J_{\min}} \right\rceil,$$

q is assigned to the corresponding representative. Otherwise, no existing representative can satisfy the threshold, and q becomes a new representative.

Unlike MinHash, KSSD sketches are not fixed-length. Therefore, RabbitTClust2 estimates the Jaccard similarity between q and each candidate representative r as

$$\widehat{J} = \frac{c}{|S_q| + |S_r| - c}.$$

Equivalently, representative r can reach $J_{\min}$ only if

$$c \geq \left\lceil \frac{J_{\min}(|S_q| + |S_r|)}{1 + J_{\min}} \right\rceil.$$

RabbitTClust2 first finds the representative with the largest $\widehat{J}$. If this largest value is at least $J_{\min}$, q is assigned to that representative. Otherwise, q becomes a new representative and its sketch hashes are inserted into the index.

RabbitTClust2 further uses sketch length to prune the representative index in KSSD mode. Genomes are processed in descending sketch-length order, so all existing representatives have sketches no smaller than the current query sketch. For a representative r and query q, if

$$|S_r| > |S_q| / J_{\min},$$

then even full containment of S_q in S_r cannot make their Jaccard similarity reach $J_{\min}$. Because later queries have sketches no larger than S_q, such a representative cannot match the current query or any later query. RabbitTClust2 therefore removes the representative from the search index, while still keeping it as a cluster representative in the final output.

D. MST-based clustering

The MST construction phase consists of two steps: computing sketch-based distances between genomes and constructing the MST with Kruskal's algorithm. In RabbitTClust, distances are computed by pairwise sketch intersection. This requires $O(N^2 \cdot S)$ time for N genomes and sketch size S. RabbitTClust2 replaces this exhaustive pairwise intersection step with a dataset-wide inverted index, as shown in Fig. 1(c). The inverted index maps each sketch hash to the genomes containing this hash. Thus, RabbitTClust2 only considers genome pairs that share at least one sketch hash as candidate pairs. Distances are computed only for these candidate pairs, rather than for all genome pairs. Building the inverted index takes $O(N \cdot s)$ time, where s is the average number of hash values in a sketch. RabbitTClust2 scans the posting lists in the index to count hash co-occurrences between genome pairs. These co-occurrence counts are used as sketch intersection sizes for distance estimation. Let v be the average number of genomes stored under one hash value. The index scan costs $O(v \cdot N \cdot s)$, so the total cost of index construction and co-occurrence counting is $O((v+1) \cdot N \cdot s)$. Since s is comparable to S and $v \ll N$ in sparse sketch spaces, the inverted-index-based method is much cheaper than the $O(N^2 \cdot S)$ pairwise intersection used in RabbitTClust. RabbitTClust2 constructs the inverted index during sketch generation. For each genome, its sketch hashes are inserted into the index as soon as the sketch is produced. This avoids a second scan over all sketches and improves data locality.

RabbitTClust2 further applies a threshold-aware pruning rule before distance evaluation. Using the minimum Jaccard threshold $J_{\min}$ corresponding to the user-specified Mash distance threshold (Eq. 1), RabbitTClust2 removes candidate pairs whose maximum possible Jaccard similarity is still below $J_{\min}$. For variable-length sketches such as KSSD, RabbitTClust2 uses a length-ratio bound. For a sketch pair (i, j), if

$$\frac{\min(|S_i|, |S_j|)}{\max(|S_i|, |S_j|)} < J_{\min},$$

then even full containment of the smaller sketch in the larger sketch cannot reach the required Jaccard similarity. The pair is therefore safely skipped before distance evaluation. RabbitTClust2 computes sketch distances only for the remaining candidate pairs and uses these distances in Kruskal's algorithm to construct the MST.

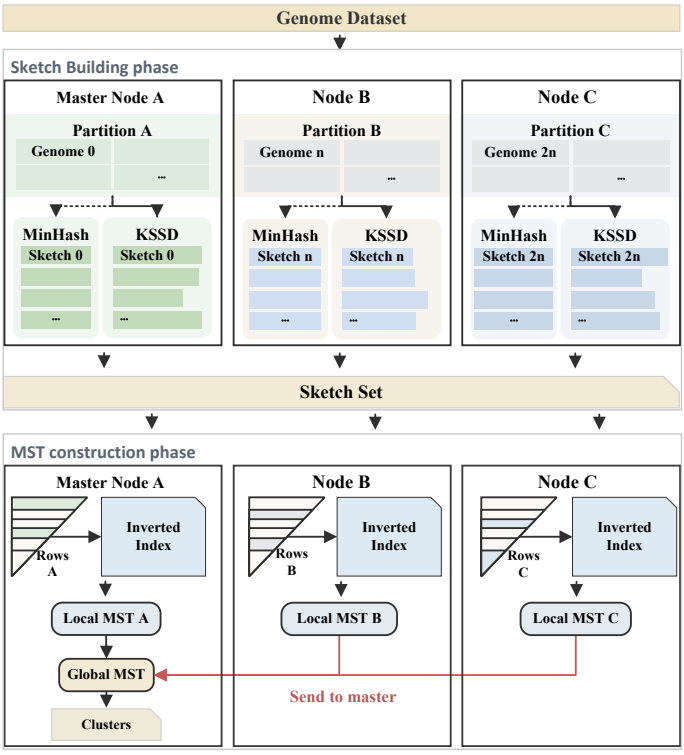


Fig. 2. Distributed implementation of MST mode for large-scale genome clustering.

As genome databases grow, MST-based clustering becomes increasingly limited by single-server resources, such as I/O bandwidth. To address this challenge, RabbitTClust2 provides

an MPI-based distributed implementation across multiple compute nodes. Sketch construction is naturally parallel because each genome can be processed independently. RabbitTClust2 therefore partitions input genomes evenly across nodes, and each node builds KSSD or MinHash sketches for its assigned genomes. After local sketch construction, the generated sketches are written to the shared file system. Because sketches are much smaller than raw genome sequences, this aggregation introduces limited overhead and does not require fine-grained inter-node communication. For MST-based clustering, RabbitTClust2 uses distributed distance evaluation. Each node loads the complete sketch set and builds a local inverted index. RabbitTClust2 employs a cyclic distribution strategy for rows in the lower-triangular distance matrix. This strategy improves load balance when the computational cost of distance calculation varies across rows. Each node computes distances for its assigned rows, generates candidate edges, and sends these edges to the master node. The master node then constructs the global MST using Kruskal's algorithm.

Greedy mode is less suitable for distributed parallelization because its assignments are order-dependent. Each new genome is assigned based on the current representative set, and this representative set changes during clustering. Therefore, later assignments depend on earlier decisions, creating a sequential dependency that limits efficient multi-node parallelization.

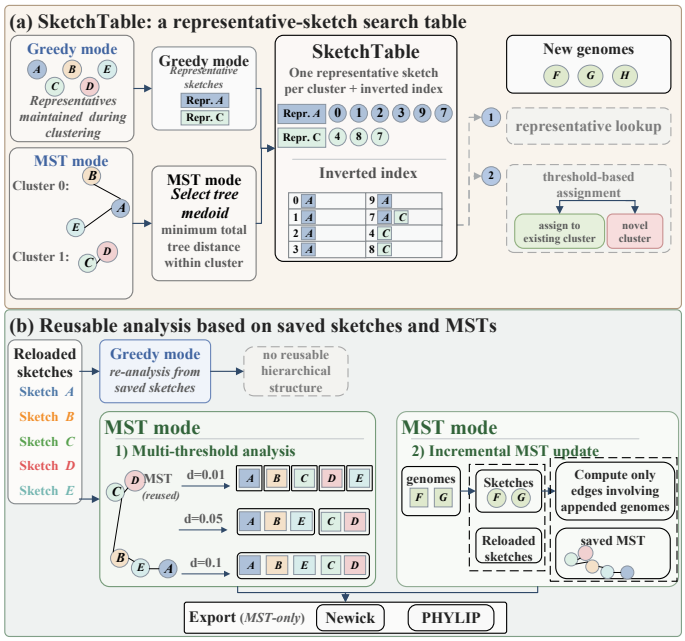


Fig. 3. Representative-sketch search table and reusable analysis in RabbitTClust2.

E. Output Files

After clustering, users often need to analyze clustering results, visualize the clustering structure, and reuse representative information in downstream tasks. RabbitTClust2 therefore outputs both standard clustering result files and a reusable representative-sketch search table. The standard outputs include genome-to-cluster assignment files, representative genomes, and CD-HIT-style cluster files. The assignment file records the cluster ID of each genome, while the CD-HIT-style file lists the member genomes and basic sequence information for each cluster. These files allow users to analyze clustering results and identify the representative genome of each cluster. In MST mode, RabbitTClust2 can also export Newick and PHYLIP files, making it easier to use the results in existing bioinformatics tools.

In addition to these result files, RabbitTClust2 can build SketchTable, a representative-sketch search table for later new-genome assignment. SketchTable stores an inverted index from sketch hashes to representative IDs. After being saved, it can be reloaded to quickly retrieve candidate representatives for new genomes. In Greedy mode, the representatives maintained during clustering are directly written into SketchTable. In MST mode, RabbitTClust2 first cuts the MST at the user-specified threshold to obtain clusters and then selects one representative for each cluster. Specifically, RabbitTClust2 uses the tree medoid, defined as the genome with the smallest total path distance to all other genomes in the same cluster:

$$r_i = \arg\min_{v \in C_i} \sum_{u \in C_i} d_T(u, v),$$

where $d_T(u, v)$ denotes the path distance between genomes u and v on the MST.

F. Incremental and Multi-Threshold Clustering

In addition to final result files, RabbitTClust2 saves intermediate structures that can be reused in later runs. In MST mode, saved sketches and MSTs are reused for incremental clustering and multi-threshold clustering. Incremental clustering updates an existing genome database after new genomes are added, while multi-threshold clustering generates clustering results under different distance thresholds.

RabbitTClust2 supports incremental clustering through the `--append` option (Fig. 3(b)). Given an existing dataset A and an appended set B, RabbitTClust2 reuses the saved sketches and MST of A, and constructs sketches only for genomes in B. The sketch hashes of genomes in B are then inserted into the existing inverted index. Using the updated index, RabbitTClust2 retrieves candidate pairs involving the appended genomes, including B–A pairs and index-retrieved B–B pairs. Distances are computed only for these new candidate pairs, rather than for all pairs in $A \cup B$. The resulting candidate edges are combined with the saved MST edges of A to construct the updated MST for $A \cup B$. This process avoids recomputing old A–A distances and avoids rebuilding the full clustering workflow from scratch.

RabbitTClust2 also supports multi-threshold clustering in MST mode by reusing a precomputed MST (Fig. 3(b)). Threshold selection often requires evaluating clustering results under multiple distance cutoffs. Without reuse, the MST must be rebuilt for each cutoff, which is expensive on large genome databases. RabbitTClust2 avoids this repeated computation by loading the precomputed MST and cutting the same MST

at different thresholds. Thus, multi-threshold clustering is reduced to one MST construction followed by low-cost threshold cuts.

III. Results

We benchmark RabbitTClust2 against RabbitTClust and four widely used tools (MeShClust3 v3.0, Gclust v1.0, Mash v2.3, and Mothur v1.48.3) to evaluate efficiency and clustering quality on large bacterial genome datasets. Unless otherwise stated, all sketch-based comparisons use a k-mer size of 21. For distance-threshold-based clustering, we use a Mash-distance threshold of 0.05, corresponding to the commonly used species-level ANI boundary of approximately 95%. Datasets are summarized in Table I. We use two NCBI bacterial genome datasets: GenBank bacteria (Gen-Bac, Release 270; 2,841,651 genomes; 11 TB) and RefSeq bacteria (Ref-Bac, Release 233; 208,753 genomes; 881 GB). For the incremental-update experiments, we randomly sample 1,000 genomes from Gen-Bac as the update set, denoted as Sub-Bac, and use the Ref-Bac as the base database. To evaluate performance on modern x86 servers with different SIMD instruction-set support, we test RabbitTClust2 on an Intel server with AVX-512 and an AMD server with AVX2, and further evaluate the distributed implementation on a 16-node cluster (Table II).

We evaluate clustering accuracy against reference taxonomic labels on Ref-Bac. Species-level labels are taken directly from the `species_taxid` field in `assembly_summary_refseq`, while genus-level labels are obtained by mapping each `species_taxid` to its genus ancestor in the NCBI taxonomy dump (e.g., `nodes.dmp`). We report Normalized Mutual Information (NMI) and pairwise precision, recall, and F1-score at the species level. For genome pairs (i, j), true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN) are defined based on predicted cluster membership and the reference taxonomic labels. We compute these pairwise counts from the pair confusion matrix between predicted clusters and reference labels, without explicitly enumerating all $O(N^2)$ genome pairs, and derive precision, recall, and F1-score accordingly.

TABLE I
Description of datasets.

Dataset	Description	Genomes	Data Size (plain)
Gen-Bac	the bacterial genome dataset from GenBank Release 270	2,841,651	11 TB
Ref-Bac	the bacterial genome dataset from RefSeq Release 233	208,753	881 GB
Sub-Bac	the subset of Gen-Bac	1,000	4.5 GB

TABLE II
Hardware and software configurations.

Environment	AMD Server	Intel Server	Distributed Platform
Nodes	1	1	16
CPU	AMD EPYC 7T83	Intel Xeon Platinum 8375C	AMD EPYC 9755
Cores per node	128	128	256
RAM	256 GB	256 GB	768 GB
MPI Library	N/A	N/A	Intel MPI 2024.1
Interconnect	N/A	N/A	100 Gb/s EDR InfiniBand

TABLE III
Performance comparison on the Gen-Bac dataset.

Platform	Tool	Memory (GB)	Sketch (s)	Clustering (s)	Total (s)
Intel	RabbitTClust-MST	192.0	3220.5	299200.0	302420.5
	RabbitTClust-Greedy	142.3	3225.7	9560.2	12785.9
	RabbitTClust2-MH+MST	131.0	2143.1	71034.0	73177.1
	RabbitTClust2-MH+Greedy	114.5	2156.0	803.3	2959.3
	RabbitTClust2-KSSD+MST	98.2	1329.2	70494.8	71824.0
	RabbitTClust2-KSSD+Greedy	87.6	1309.6	879.1	2188.7
AMD	RabbitTClust-MST	207.0	3031.9	270134.0	273165.9
	RabbitTClust-Greedy	147.3	3026.5	6195.3	9221.8
	RabbitTClust2-MH+MST	117.6	2269.4	58403.0	60672.4
	RabbitTClust2-MH+Greedy	101.2	2267.0	849.1	3116.1
	RabbitTClust2-KSSD+MST	90.6	1994.8	56306.4	58301.2
	RabbitTClust2-KSSD+Greedy	85.4	1996.1	1268.3	3264.4

A. Single-Server Performance Evaluation of RabbitTClust2

We first evaluate RabbitTClust2 on the 11 TB Gen-Bac dataset to test its performance on database-scale genome datasets. General-purpose baselines such as MeShClust3, Gclust, and Mothur&Mash are not practical at this scale because of memory or runtime limits. Therefore, Table III focuses on the comparison between RabbitTClust and RabbitTClust2. Across both Intel and AMD platforms, RabbitTClust2 reduces both total runtime and memory usage compared with RabbitTClust. On Intel, RabbitTClust2-KSSD+Greedy is the fastest method, finishing in 2188.7 s and using 87.6 GB of memory. Compared with RabbitTClust-MST and RabbitTClust-Greedy, this achieves 138.2× and 5.8× speedups, respectively. On AMD, RabbitTClust2-MH+Greedy is the fastest method, finishing in 3116.1 s. Compared with RabbitTClust-MST and RabbitTClust-Greedy, it achieves 87.7× and 3.0× speedups, respectively. RabbitTClust2-KSSD+Greedy uses the least memory on AMD, with a peak memory usage of 85.4 GB. These results show that RabbitTClust2 can cluster an 11 TB bacterial genome database on a single server within about one hour. These results also show that Greedy mode is already efficient for one-off clustering on Gen-Bac. Thus, its distributed execution is less critical. In contrast, MST mode is much more expensive because it needs to generate candidate genome pairs and construct a global MST. Therefore, distributed acceleration is mainly useful for the MST workflow.

We then evaluate both performance and clustering quality on Ref-Bac. Table IV summarizes the results on both Intel

TABLE IV
PERFORMANCE COMPARISON OF RABBITTCLUST2 ON THE REF-BAC DATASET.

Platform	Tool	NMI	Recall	Precision	F1	Memory	Sketch	Clustering	Total
Intel	MeShClust3	-	-	-	-	OOM	-	-	-
	Gclust	-	-	-	-	OOM	-	-	-
	Mothur&Mash	0.9354	0.6962	0.8955	0.7834	21.3 GB	7651.0 s	92764.0 s	100415.0 s
	RabbitTClust-MST	0.9518	0.9977	0.8441	0.9144	27.7 GB	260.3 s	1243.4 s	1503.8 s
	RabbitTClust-Greedy	0.9465	0.9981	0.8425	0.9138	13.5 GB	273.5 s	155.4 s	428.9 s
	RabbitTClust2-MH+MST	0.9525	0.9976	0.8443	0.9145	19.0 GB	166.2 s	66.7 s	232.9 s
	RabbitTClust2-MH+Greedy	0.9465	0.9981	0.8425	0.9138	14.8 GB	167.7 s	76.1 s	243.7 s
	RabbitTClust2-KSSD+MST	0.9518	0.9976	0.8439	0.9143	10.0 GB	100.8 s	61.1 s	161.9 s
	RabbitTClust2-KSSD+Greedy	0.9560	0.9932	0.8765	0.9312	8.5 GB	100.5 s	62.7 s	163.2 s
AMD	MeShClust3	-	-	-	-	OOM	-	-	-
	Gclust	-	-	-	-	OOM	-	-	-
	Mothur&Mash	0.9354	0.6962	0.8955	0.7834	17.9 GB	6535.0 s	101008.0 s	107543.0 s
	RabbitTClust-MST	0.9518	0.9977	0.8441	0.9144	26.3 GB	232.8 s	607.3 s	840.1 s
	RabbitTClust-Greedy	0.9465	0.9981	0.8425	0.9138	14.1 GB	243.3 s	96.0 s	339.3 s
	RabbitTClust2-MH+MST	0.9525	0.9976	0.8443	0.9145	18.8 GB	183.5 s	58.5 s	242.0 s
	RabbitTClust2-MH+Greedy	0.9465	0.9981	0.8425	0.9138	15.8 GB	182.8 s	97.8 s	280.7 s
	RabbitTClust2-KSSD+MST	0.9518	0.9976	0.8439	0.9143	9.1 GB	161.3 s	61.3 s	222.4 s
	RabbitTClust2-KSSD+Greedy	0.9560	0.9932	0.8765	0.9312	8.2 GB	160.0 s	71.2 s	231.3 s

TABLE V
PERFORMANCE COMPARISON OF RABBITTCLUST2 ON THE SUB-BAC DATASET.

Platform	Tool	NMI	Recall	Precision	F1	Memory	Sketch	Clustering	Total
Intel	MeShClust3	0.8944	0.0443	0.9887	0.0847	46.4 GB	-	-	12958.0 s
	Gclust	0.8926	0.0341	0.9890	0.0660	5.6 GB	-	-	9674.0 s
	Mothur&Mash	0.9885	0.9964	0.9036	0.9478	0.8 GB	3.1 s	0.6 s	3.7 s
	RabbitTClust-MST	0.9876	0.9964	0.8899	0.9402	1.5 GB	2.0 s	0.2 s	2.2 s
	RabbitTClust-Greedy	0.9844	0.9986	0.8659	0.9275	1.5 GB	2.2 s	0.2 s	2.4 s
	RabbitTClust2-MH+MST	0.9876	0.9959	0.8901	0.9401	1.6 GB	1.1 s	0.5 s	1.6 s
	RabbitTClust2-MH+Greedy	0.9844	0.9986	0.8659	0.9275	2.3 GB	1.0 s	0.8 s	1.8 s
	RabbitTClust2-KSSD+MST	0.9873	0.9973	0.8866	0.9387	1.2 GB	1.5 s	0.2 s	1.7 s
	RabbitTClust2-KSSD+Greedy	0.9885	0.9964	0.9036	0.9478	1.8 GB	1.1 s	0.6 s	1.7 s
AMD	MeShClust3	0.8944	0.0443	0.9887	0.0847	49.1 GB	-	-	7116 s
	Gclust	0.8926	0.0341	0.9890	0.0660	5.9 GB	-	-	15680.0 s
	Mothur&Mash	0.9885	0.9964	0.9036	0.9478	1.3 GB	3.3 s	0.5 s	3.8 s
	RabbitTClust-MST	0.9876	0.9964	0.8899	0.9402	1.5 GB	2.1 s	0.2 s	2.3 s
	RabbitTClust-Greedy	0.9844	0.9986	0.8659	0.9275	1.5 GB	2.3 s	0.2 s	2.5 s
	RabbitTClust2-MH+MST	0.9876	0.9959	0.8901	0.9401	1.6 GB	1.0 s	0.8 s	1.8 s
	RabbitTClust2-MH+Greedy	0.9844	0.9986	0.8659	0.9275	1.2 GB	1.0 s	0.8 s	1.8 s
	RabbitTClust2-KSSD+MST	0.9873	0.9973	0.8866	0.9387	2.1 GB	1.5 s	0.4 s	1.9 s
	RabbitTClust2-KSSD+Greedy	0.9885	0.9964	0.9036	0.9478	1.5 GB	1.2 s	0.5 s	1.7 s

and AMD servers. MeShClust3 and Gclust fail with out-of-memory errors on this dataset, and Mothur&Mash completes but is much slower than RabbitTClust2. Across both platforms, RabbitTClust2 reduces runtime while maintaining comparable or higher clustering quality. Under the default setting of $k = 21$ and $d = 0.05$, KSSD+Greedy achieves the best clustering quality on Ref-Bac, with NMI$= 0.9560$ and F1$= 0.9312$. It also has the smallest memory footprint among the tested RabbitTClust2 methods, using 8.5 GB on Intel and 8.2 GB on AMD. Its total runtime is close to KSSD+MST, while giving higher F1 and using less memory. Compared with RabbitTClust, KSSD+Greedy gives clear acceleration. On Intel, it finishes in 163.2 s, giving 9.2× and 2.6× speedups over RabbitTClust-MST and RabbitTClust-Greedy, respectively. On AMD, it finishes in 231.3 s, giving 3.6× and 1.5× speedups over the same two baselines.

Finally, we evaluate all methods on Sub-Bac to provide a smaller-scale comparison where more baselines can finish. Table V reports the results. On this reduced dataset, RabbitTClust2 and RabbitTClust have closer runtimes because fixed overheads account for a larger fraction of the total runtime. Even so, RabbitTClust2 remains among the fastest methods and maintains clustering quality comparable to the best-performing alternatives. MeShClust3 and Gclust can finish on Sub-Bac, but their recall and F1 are much lower than the other methods. Mothur&Mash achieves good clustering quality on Sub-Bac, but it is still slower than the fastest RabbitTClust2 method. These results show that RabbitTClust2 performs well across different dataset sizes, with stronger benefits on large genome databases.

B. Distributed Performance Evaluation of RabbitTClust2

We first evaluate the distributed performance of RabbitTClust2 on the TB-scale Gen-Bac dataset. Table VI reports the performance of the MPI implementation using 16 nodes and 4096 CPU cores. KSSD+MST takes 1729.6 s in total, including 345.2 s for sketch construction, 287.4 s for index construction, and 1097.0 s for clustering. MinHash+MST takes

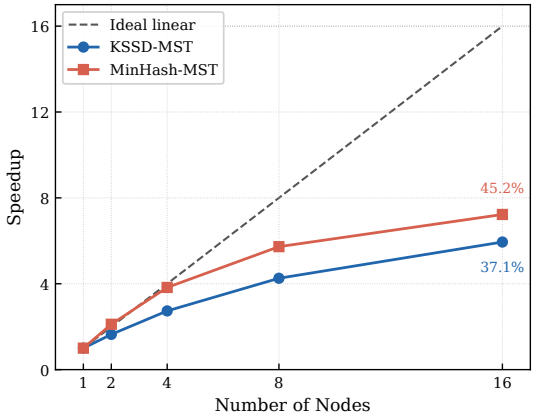


Fig. 4. Strong scalability of RabbitTClust2 on the Distributed Platform.

TABLE VI
PERFORMANCE OF MPI IMPLEMENTATION ON THE GEN-BAC DATASET USING 16 NODES (4096 CORES).

Tool	Sketch	Index	Cluster	Total
RabbitTClust2-KSSD+MST	345.2 s	287.4 s	1097.0 s	1729.6 s
RabbitTClust2-MH+MST	473.5 s	102.8 s	1062.6 s	1638.9 s

1638.9 s in total, including 473.5 s for sketch construction, 102.8 s for index construction, and 1062.6 s for clustering. Both KSSD+MST and MinHash+MST finish clustering the 11 TB dataset in less than 30 minutes. This shows that RabbitTClust2 can efficiently perform MST-based clustering on TB-scale genome datasets in a distributed environment.

We then evaluate strong scalability on Ref-Bac by increasing the number of nodes from 1 to 16. As shown in Fig. 4, both KSSD+MST and MinHash+MST obtain clear speedups as more nodes are used. At 16 nodes, KSSD+MST reaches a speedup of about 5.9×, corresponding to 37.1% parallel efficiency. MinHash+MST reaches a higher speedup of about 7.2×, corresponding to 45.2% parallel efficiency. However, neither method follows ideal linear scaling. RabbitTClust2 parallelizes sketch construction and MST construction, but the inverted index is still built independently on each node. This repeated index construction becomes a larger overhead when more nodes are used. As a result, the speedup improves with more nodes but does not follow ideal linear scaling.

C. Performance Evaluation of Incremental Clustering

We next evaluate RabbitTClust2 for incremental MST updating by adding Sub-Bac to an existing Ref-Bac database. Table VII reports the runtime of updating the clustering after adding Sub-Bac. RabbitTClust does not support sketch reuse or incremental MST updating. Therefore, it is used as a full re-clustering baseline. It reprocesses the whole dataset, taking 294.8 s for sketch construction and 813.2 s for clustering, with a total runtime of 1108.0 s. RabbitTClust2 avoids this full reconstruction. It reuses the saved sketches and MST of the base set, constructs sketches only for the appended genomes, and computes only the new candidate distances

TABLE VII
PERFORMANCE COMPARISON FOR INCREMENTAL CLUSTERING.

Tool	Sketch	Clustering	Total
RabbitTClust-MST	294.8 s	813.2 s	1108.0 s
RabbitTClust2-KSSD+MST	1.0 s	17.4 s	18.4 s
RabbitTClust2-MH+MST	1.1 s	16.2 s	17.3 s

TABLE VIII
PERFORMANCE COMPARISON FOR MULTI-THRESHOLD CLUSTERING.

Tool	#Thresholds	Total
RabbitTClust-MST	[0.01, 0.20] (n=20)	27569.3 s
RabbitTClust-Greedy	[0.01, 0.20] (n=20)	5473.5 s
RabbitTClust2-MinHash+Greedy	[0.01, 0.20] (n=20)	3142.0 s
RabbitTClust2-KSSD+Greedy	[0.01, 0.20] (n=20)	2199.6 s
RabbitTClust2-MinHash+MST	[0.01, 0.20] (n=20)	243.7 s
RabbitTClust2-KSSD+MST	[0.01, 0.20] (n=20)	187.2 s

Note: #Thresholds indicates the range and number of clustering thresholds used in the experiment.

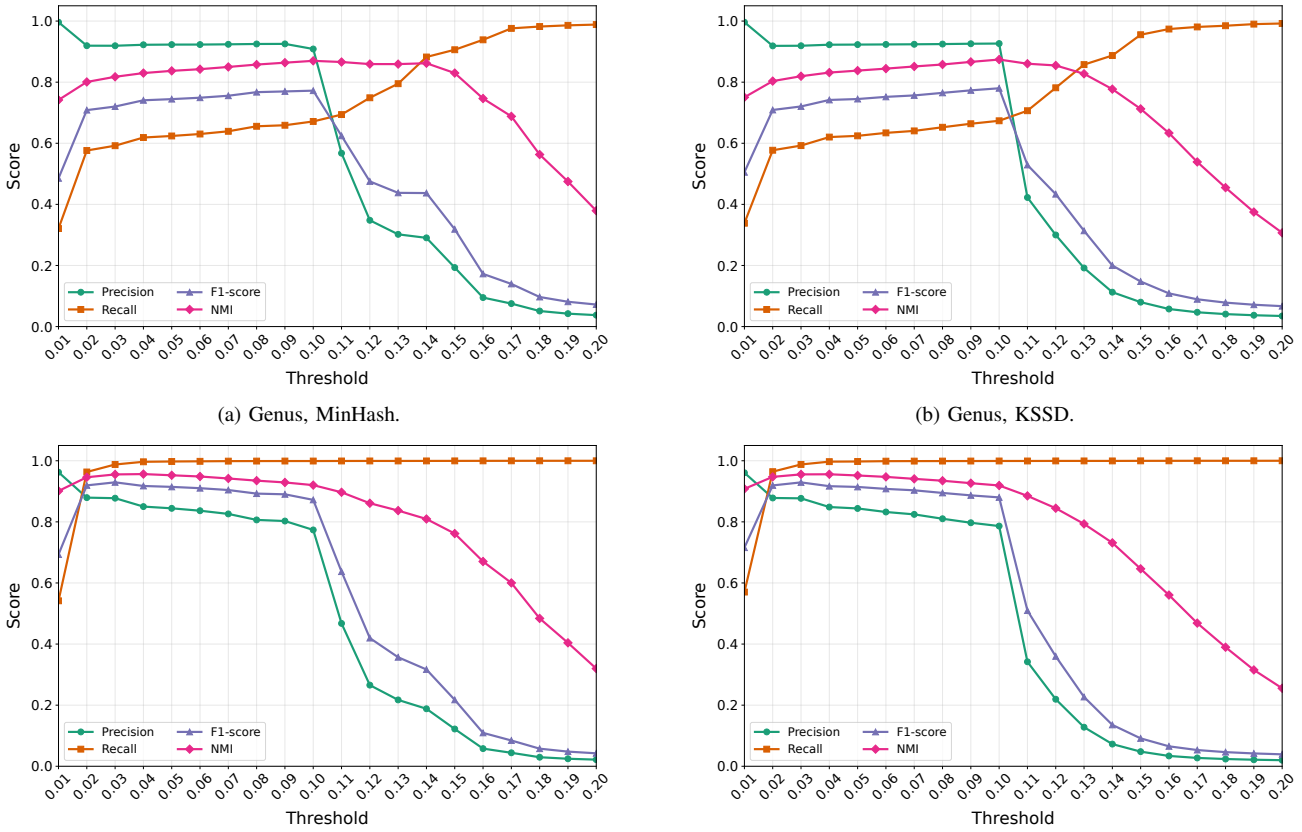
involving the appended genomes. With this incremental workflow, KSSD+MST finishes in 18.4 s and MinHash+MST finishes in 17.3 s. This corresponds to 60.2× and 64.0× speedups over full re-clustering with RabbitTClust-MST, respectively. These results show that RabbitTClust2 can add new genomes to an existing clustering result with much lower cost than rebuilding the clustering from scratch.

D. Performance Evaluation of Multi-Threshold Clustering

We next evaluate multi-threshold clustering on Ref-Bac using 20 distance thresholds from 0.01 to 0.20. Table VIII reports the total runtime. RabbitTClust-MST takes 27569.3 s, or about 7.7 h, because it reruns the clustering workflow for each threshold. RabbitTClust-Greedy takes 5473.5 s, while RabbitTClust2 reduces the greedy-mode runtime to 2199.6 s with KSSD+Greedy and 3142.0 s with MinHash+Greedy. However, greedy-based methods still need to rerun clustering at each threshold because they do not maintain a reusable structure. The MST mode is much more efficient for this task. RabbitTClust2-KSSD+MST completes the 20-threshold analysis in 187.2 s, and RabbitTClust2-MinHash+MST completes it in 243.7 s. Compared with RabbitTClust-MST, KSSD+MST achieves a 147.3× speedup. It is also 11.8× faster than RabbitTClust2-KSSD+Greedy, the fastest greedy method in this experiment. The speedup mainly comes from MST reuse. RabbitTClust2 builds the MST once using the largest threshold in the sweep. It then obtains clustering results at smaller thresholds by cutting the same MST, instead of rebuilding the clustering structure for each threshold. Therefore, the total cost is dominated by one MST construction plus multiple inexpensive MST cuts. These results show that MST reuse makes multi-threshold clustering practical on large genome datasets.

E. Multi-Threshold Analysis of Species- and Genus-Level Boundary Cases

To investigate whether genome similarity shows clear species- and genus-level boundaries, we use RabbitTClust2 in



(a) Genus, MinHash.

(b) Genus, KSSD.

(c) Species, MinHash.

(d) Species, KSSD.

Fig. 5. Clustering performance of RabbitTClust2 under different distance thresholds.

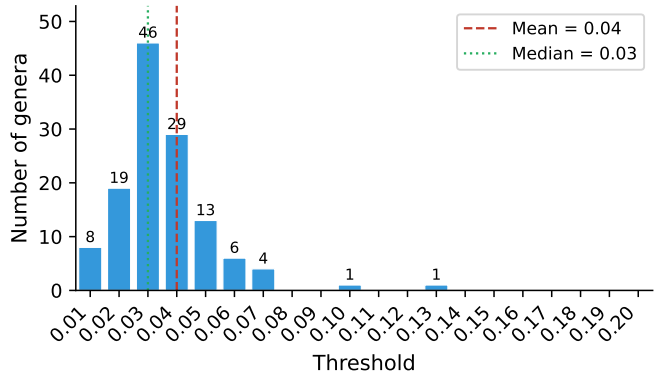
TABLE IX
REPRESENTATIVE TAXONOMIC MERGING CASES SUGGESTED BY THE MULTI-THRESHOLD ANALYSIS.

Level	Label A	Label B	Cluster size	A in cluster	B in cluster	A covered	B covered	Frac. A	Frac. B	Threshold
Genus	Escherichia	Shigella	19763	18000	1763	18342	1764	0.981	0.999	0.03
Species	***Campylobacter upsaliensis* DSM 5365**	***Campylobacter felis***	107	101	6	113	6	0.894	1.000	0.02
Species	***Caldibacillus thermoamylovorans***	***Caldifermentibacillus hisashii***	22	6	16	6	16	1.000	1.000	0.02
Species	***Xanthomonas perforans* 91-118**	***Xanthomonas euvesicatoria* pv. *polysciadis***	186	97	58	97	58	1.000	1.000	0.02
Species	*Yersinia pestis* subsp. *microtus* bv. *Xilingolensis*	*Yersinia pseudotuberculosis* B-7194	404	341	60	341	60	1.000	1.000	0.02

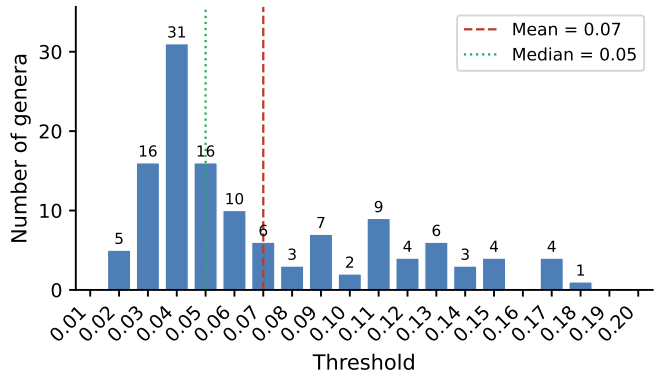
TABLE X
REPRESENTATIVE INTRA-LABEL SPLITTING CASES DETECTED BY THE MULTI-THRESHOLD ANALYSIS.

Level	Label	Covered genomes	Ground-truth genomes	No. of clusters	Threshold	Label-specific cluster sizes and purities
Genus	***Enterovibrio***	44	62	4	0.18	28 (1.000), 13 (1.000), 2 (1.000), 1 (1.000)
Genus	***Agarivorans***	16	25	3	0.18	9 (0.900), 6 (1.000), 1 (1.000)
Genus	*Helicobacter*	1585	3691	28	0.18	972 (1.000), 468 (1.000), 75 (1.000), 17 (1.000), 14 (1.000), ...
Genus	*Prochlorococcus*	80	158	12	0.18	35 (1.000), 17 (1.000), 13 (1.000), 5 (1.000), 3 (1.000), ...
Species	***uncultured Dubosiella* sp.**	82	106	2	0.10	41 (1.000), 41 (1.000)
Species	***Anaerorhabdus* sp.**	16	19	3	0.10	8 (1.000), 6 (1.000), 2 (1.000)
Species	"Candidatus *Karelsulcia muelleri*" PSPU	91	93	5	0.10	31 (1.000), 28 (1.000), 20 (1.000), 10 (1.000), 2 (1.000)

MST mode to evaluate KSSD- and MinHash-based clustering on Ref-Bac across Mash-distance thresholds from 0.01 to 0.20, using NCBI taxonomic labels at both the species and genus levels as references (Fig. 5). At both taxonomic levels,



(a) F1-maximizing threshold distribution



(b) Knee threshold distribution

Fig. 6. (**a**) F1-maximizing threshold and (**b**) knee threshold distribution for genera with clear within-genus transition patterns.

increasing the threshold from 0.01 to 0.02 produces a similar trade-off: precision decreases sharply, while recall, F1-score, and NMI increase. This result indicates that a threshold of 0.01 is too strict, causing many genomes from the same taxon to be split into different clusters. As the threshold becomes more relaxed, more within-taxon genomes are grouped together, but genomes from different taxa are also more likely to be merged into the same clusters. At the genus level, the curves show a clear transition pattern. From 0.02 to 0.10, precision remains high and recall increases steadily, indicating that more genomes from the same genus are gradually connected while cross-genus merging remains limited. However, many genomes from the same genus remain separated at 0.10. When the threshold increases from 0.10 to 0.11, recall rises abruptly and precision begins to decrease. This range marks a genus-level transition region: more genomes from the same genus are connected, but cross-genus merging also becomes more frequent. By contrast, the species-level curves do not show a clear global transition. Recall stabilizes above 0.99 after the threshold exceeds 0.03, but precision remains relatively low across the scanned range. This result suggests that species-level genome similarity boundaries are not uniform across the dataset. Some species are well separated from their closest relatives, while others are highly similar to neighboring species and are harder to distinguish. As a result, the whole-dataset species-level curve mixes these different species-level behaviors and can hide clear transition patterns.

We therefore analyze species-level threshold responses within individual genera. For each genus, we first identify the threshold that gives the highest F1-score. We then examine the post-peak region to detect whether the threshold-response curve shows a sharp transition. Specifically, we compute one-step changes in F1 between consecutive thresholds and convert them into robust z-scores using the median absolute deviation (MAD). A genus is considered to have a clear transition if a post-peak change exceeds the empirical robust z-score cutoff of 3.5, with recall already reaching the plateau and precision showing a decreasing trend. The knee threshold is defined as the first post-peak threshold where this transition occurs. Among the 316 genera with at least 100 genomes, 127 genera (40.2%) show a clear within-genus species-level transition pattern. For these genera, the thresholds that maximize F1-score are concentrated at low values, with a median of 0.03 and a mean of 0.04. This result indicates that a Mash-distance range of 0.03–0.04 provides a practical operating range for within-genus species clustering on Ref-Bac (Fig. 6a). In contrast, the knee thresholds are more widely distributed, ranging from 0.02 to 0.18, with a median of 0.05 and a mean of 0.07 (Fig. 6b). Some genera show knee thresholds below 0.03, while others show transition points only at much more relaxed thresholds above 0.15. This wide distribution suggests that species-level boundaries inferred from genome similarity do not always align perfectly with NCBI taxonomic labels. In some genera, genomes assigned to the same species may require relaxed thresholds to be grouped together, while genomes assigned to different species may merge at more strict thresholds.

F. Threshold-Dependent Taxonomic Label Splitting and Merging

Using the MST-mode multi-threshold results on Ref-Bac, we further examine cases in which similarity-based clusters do not fully align with NCBI taxonomic labels. These include genomes with the same label that remain separated even under relaxed thresholds (intra-label splitting), and genomes with different labels that are grouped together even under strict thresholds (taxonomic merging). For intra-label splitting, we define relaxed thresholds as 0.10 at the species level and 0.18 at the genus level, because species and genus labels span different distance ranges. Stability is checked over nearby thresholds, i.e., 0.10–0.12 for species and 0.18–0.20 for genus. Each retained split cluster must be label-pure, with the target label accounting for at least 80% of genomes in that cluster. The split is considered stable only if the corresponding groups remain separated within the threshold window, with at least 80% group overlap between neighboring thresholds. A taxonomic merging case is a pair of different taxonomic labels that form a stable mixed cluster at a strict threshold ($d < 0.03$ for species-level labels and $d < 0.05$ for genus-level labels). For each mixed cluster, the two dominant labels must together account for at least 80% of the cluster. For the high-confidence examples reported in Table IX, we further require both labels to have high cluster coverage, with *Frac. A* and *Frac. B* no lower than 0.70. Here, *Frac. A* and *Frac. B* denote the fractions

of covered genomes from labels A and B that are included in the mixed cluster, respectively.

In total, this analysis identifies 53 taxonomic merging pairs and 6 intra-label splitting cases at the species level, and 6 potential taxonomic merging pairs and 49 intra-label splitting cases at the genus level. The complete lists are provided in our repository. Tables IX and X show representative examples. For taxonomic merging, RabbitTClust2 identifies known boundary cases, suggesting that the screening strategy is reasonable. At the genus level, RabbitTClust2 recovers the classic *Escherichia–Shigella* mixed-label cluster, which covers 98.1% and 99.9% of the two labels, respectively. This is a well-known taxonomic boundary case, because genomic studies have long shown that *Shigella* is closely embedded within *Escherichia coli* lineages [21]–[23]. At the species level, RabbitTClust2 also recovers the known *Yersinia pestis–Yersinia pseudotuberculosis* mixed-label cluster [24], [25]. Beyond these known cases, RabbitTClust2 identifies three additional species-level mixed-label pairs: *Campylobacter upsaliensis–Campylobacter felis*, *Caldibacillus thermoamylovorans–Caldifermentibacillus hisashii*, and *Xanthomonas perforans–Xanthomonas euvesicatoria* pv. *polysciadis*. These pairs are highlighted in bold in Table IX. To further examine them, we perform all-to-all FastANI [26] comparisons within each mixed-label cluster. The three clusters have high ANI values, with mean ANI values of 97.56%, 98.20%, and 98.84%, and minimum ANI values of 95.31%, 97.22%, and 97.47%. These ANI values are all above the commonly used species boundary of about 95% ANI, suggesting that these mixed-label clusters may be worth further examination.

For intra-label splitting, RabbitTClust2 also recovers known within-label structure. For example, *Helicobacter* and *Prochlorococcus* are known to contain clearly separated internal groups [27], [28], suggesting that the detected splits can reflect genuine sequence divergence within a taxonomic label. At the species-label level, "Candidatus *Karelsulcia muelleri*" PSPU is also split into several pure groups. This is consistent with the known pattern that endosymbiotic bacteria can form distinct host-associated lineages [29]. The same screen also identifies additional intra-label splitting cases, highlighted in bold in Table X. At the genus level, *Enterovibrio* is separated into two major high-purity groups of 28 and 13 genomes, and *Agarivorans* is mainly separated into groups of 9 and 6 genomes. At the species-label level, *uncultured Dubosiella* sp. is split into two pure groups of 41 genomes each, and *Anaerorhabdus* sp. is split into groups of 8, 6, and 2 genomes. Cross-cluster FastANI comparisons further support these splits. We further examine the intra-label splitting cases using FastANI comparisons between split groups only. For *Enterovibrio*, the reported between-group ANI values are low, with mean ANI values of 78.4%–80.0% and a maximum ANI of 80.3%. *Agarivorans* shows a similar pattern, with mean ANI values of 79.7%–80.3% between split groups and a maximum ANI of 80.6%. For the species-label cases, *uncultured Dubosiella* sp. has a mean between-group ANI of 78.3% and a maximum ANI of 79.3% between its two 41-genome groups, while *Anaerorhabdus* sp. has mean between-group ANI values of 79.5%–81.2% and a maximum of 81.7%. These low between-group ANI values indicate substantial sequence-similarity differences between the split groups, suggesting that these intra-label splits deserve further analysis.

IV. Conclusion

In this paper, we presented RabbitTClust2, a sketch-based genome clustering framework for large-scale genome databases. By combining compact KSSD or MinHash sketches with inverted-index-based candidate retrieval and threshold-aware pruning, RabbitTClust2 reduces unnecessary genome comparisons while preserving clustering quality. Its Greedy mode provides an efficient solution for one-off clustering on very large datasets, whereas its MST mode supports reusable analysis for incremental updates and multi-threshold clustering. RabbitTClust2 also supports distributed MST construction for large-scale datasets, and provides multiple output formats and representative-sketch search tables for genome assignment. Experiments on large bacterial genome datasets demonstrate that RabbitTClust2 improves runtime and memory efficiency over existing methods while maintaining high clustering accuracy. In particular, its reusable MST-based workflow reduces the cost of database updates and threshold exploration, enabling multi-threshold analysis beyond a single clustering run. The multi-threshold results further show that genome-similarity-based clusters can reveal threshold-dependent patterns and potential differences from existing taxonomic labels. Future work will focus on further improving distributed index construction, supporting more efficient dynamic updates, and extending the framework to broader genome database management scenarios. In addition, integrating similarity-based clustering with taxonomic labels, epidemiological context, and other metadata may provide more interpretable clustering results for downstream biomedical and public-health applications. RabbitTClust2 is available at `https://github.com/RabbitBio/RabbitTClust`.

Conflict of Interest

None of the authors have a conflict of interest to disclose.

References

[1] T. Zhang, X. Xu, Z. Yin, L. Yan, Y. Yang, Y. Gao, X. Duan, B. Schmidt, and W. Liu, "Rabbittclust2: Fast, scalable, and versatile clustering for massive genomic datasets," in *2025 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 2025, pp. 1495–1502.

[2] W. Li, L. Fu, B. Niu, S. Wu, and J. Wooley, "Ultrafast clustering algorithms for metagenomic sequence analysis," *Briefings in bioinformatics*, vol. 13, no. 6, pp. 656–668, 2012.

[3] Z. Zhou, J. Charlesworth, and M. Achtman, "Hiercc: a multi-level clustering scheme for population assignments based on core genome mlst," *Bioinformatics*, vol. 37, no. 20, pp. 3645–3646, 2021.

[4] P.-A. Chaumeil, A. J. Mussig, P. Hugenholtz, and D. H. Parks, "Gtdb-tk v2: memory friendly classification with the genome taxonomy database," *Bioinformatics*, vol. 38, no. 23, pp. 5315–5316, 2022.

[5] B. Schmidt and A. Hildebrandt, "Next-generation sequencing: big data meets high performance computing," *Drug discovery today*, vol. 22, no. 4, pp. 712–717, 2017.

[6] T. Goldfarb, V. K. Kodali, S. Pujar, V. Brover, B. Robbertse, C. M. Farrell, D.-H. Oh, A. Astashyn, O. Ermolaeva, D. Haddad *et al.*, "Ncbi refseq: reference sequence standards through 25 years of curation and annotation," *Nucleic Acids Research*, vol. 53, no. D1, pp. D243–D257, 2025.

[7] T. F. Smith, M. S. Waterman *et al.*, "Identification of common molecular subsequences," *Journal of molecular biology*, vol. 147, no. 1, pp. 195–197, 1981.

[8] L. Fu, B. Niu, Z. Zhu, S. Wu, and W. Li, "Cd-hit: accelerated for clustering the next-generation sequencing data," *Bioinformatics*, vol. 28, no. 23, pp. 3150–3152, 2012.

[9] M. Steinegger and J. Söding, "Clustering huge protein sequence sets in linear time," *Nature communications*, vol. 9, no. 1, p. 2542, 2018.

[10] Z. Ju, H. Zhang, J. Zhang, W. Xi, D. Huang, S. Feng, J. Meng, and Y. Wei, "A comparative study of sequence clustering algorithms," *Big Data Mining and Analytics*, vol. 8, no. 5, pp. 1011–1022, 2025.

[11] A. Zielezinski, S. Vinga, J. Almeida, and W. M. Karlowski, "Alignment-free sequence comparison: benefits, applications, and tools," *Genome biology*, vol. 18, no. 1, p. 186, 2017.

[12] W. P. Rowe, "When the levee breaks: a practical guide to sketching algorithms for processing the flood of genomic data," *Genome biology*, vol. 20, pp. 1–12, 2019.

[13] R. Li, X. He, C. Dai, H. Zhu, X. Lang, W. Chen, X. Li, D. Zhao, Y. Zhang, X. Han *et al.*, "Gclust: a parallel clustering tool for microbial genomic data," *Genomics, proteomics & bioinformatics*, vol. 17, no. 5, pp. 496–502, 2019.

[14] H. Z. Girgis, "Meshclust v3. 0: high-quality clustering of dna sequences using the mean shift algorithm and alignment-free identity scores," *BMC genomics*, vol. 23, no. 1, p. 423, 2022.

[15] P. Millan Arias, K. A. Hill, and L. Kari, "i delucs: a deep learning interactive tool for alignment-free clustering of dna sequences," *Bioinformatics*, vol. 39, no. 9, p. btad508, 2023.

[16] F. Alipour, K. A. Hill, and L. Kari, "Cgrclust: Chaos game representation for twin contrastive clustering of unlabelled dna sequences," *BMC genomics*, vol. 25, no. 1, p. 1214, 2024.

[17] X. Xu, Z. Yin, L. Yan, H. Zhang, B. Xu, Y. Wei, B. Niu, B. Schmidt, and W. Liu, "Rabbittclust: enabling fast clustering analysis of millions of bacteria genomes with minhash sketches," *Genome Biology*, vol. 24, no. 1, p. 121, 2023.

[18] B. D. Ondov, T. J. Treangen, P. Melsted, A. B. Mallonee, N. H. Bergman, S. Koren, and A. M. Phillippy, "Mash: fast genome and metagenome distance estimation using minhash," *Genome biology*, vol. 17, pp. 1–14, 2016.

[19] H. Yi, Y. Lin, C. Lin, and W. Jin, "Kssd: sequence dimensionality reduction by k-mer substring space sampling enables real-time large-scale datasets analysis," *Genome biology*, vol. 22, pp. 1–20, 2021.

[20] T. Zhang, Z. Yin, X. Xu, L. Yan, F. Zhu, X. Duan, B. Schmidt, and W. Liu, "Rabbitsketch: a high-performance sketching library for genome analysis," *Bioinformatics*, vol. 41, no. 5, p. btaf249, 2025.

[21] R. Lan and P. R. Reeves, "Escherichia coli in disguise: molecular origins of shigella," *Microbes and infection*, vol. 4, no. 11, pp. 1125–1132, 2002.

[22] J. P. Meier-Kolthoff, R. L. Hahnke, J. Petersen, C. Scheuner, V. Michael, A. Fiebig, C. Rohde, M. Rohde, B. Fartmann, L. A. Goodwin *et al.*, "Complete genome sequence of dsm 30083t, the type strain (u5/41t) of escherichia coli, and a proposal for delineating subspecies in microbial taxonomy," *Standards in genomic sciences*, vol. 9, no. 1, p. 2, 2014.

[23] M. A. Chattaway, U. Schaefer, R. Tewolde, T. J. Dallman, and C. Jenkins, "Identification of escherichia coli and shigella species from whole-genome sequences," *Journal of clinical microbiology*, vol. 55, no. 2, pp. 616–623, 2017.

[24] M. Achtman, K. Zurth, G. Morelli, G. Torrea, A. Guiyoule, and E. Carniel, "Yersinia pestis, the cause of plague, is a recently emerged clone of yersinia pseudotuberculosis," *Proceedings of the National Academy of Sciences*, vol. 96, no. 24, pp. 14 043–14 048, 1999.

[25] C. E. Demeure, O. Dussurget, G. Mas Fiol, A.-S. Le Guern, C. Savin, and J. Pizarro-Cerdá, "Yersinia pestis and plague: an updated view on evolution, virulence determinants, immune subversion, vaccination, and diagnostics," *Genes & Immunity*, vol. 20, no. 5, pp. 357–370, 2019.

[26] C. Jain, L. M. Rodriguez-R, A. M. Phillippy, K. T. Konstantinidis, and S. Aluru, "High throughput ani analysis of 90k prokaryotic genomes reveals clear species boundaries," *Nature communications*, vol. 9, no. 1, p. 5114, 2018.

[27] A. Smet, K. Yahara, M. Rossi, A. Tay, S. Backert, E. Armin, J. G. Fox, B. Flahou, R. Ducatelle, F. Haesebrouck *et al.*, "Macroevolution of gastric helicobacter species unveils interspecies admixture and time of divergence," *The ISME Journal*, vol. 12, no. 10, pp. 2518–2531, 2018.

[28] A. A. Larkin, S. K. Blinebry, C. Howes, Y. Lin, S. E. Loftus, C. A. Schmaus, E. R. Zinser, and Z. I. Johnson, "Niche partitioning and biogeography of high light adapted prochlorococcus across taxonomic ranks in the north pacific," *The ISME Journal*, vol. 10, no. 7, pp. 1555–1567, 2016.

[29] Y. Cao and C. H. Dietrich, "Phylogenomics of flavobacterial insect nutritional endosymbionts with implications for auchenorrhyncha phylogeny," *Cladistics*, vol. 38, no. 1, pp. 38–58, 2022.